

AI/ML Engineer Technical Assessment

Challenge 1: Domain-Specific Fine-Tuning

Submitted by: Satya Bonda

Position: Mid-Level AI/ML Engineer

1. Assessment Requirements

Task 1: Encoder-Decoder Fine-Tuning

Design and implement a software domain-specific fine-tuning pipeline for a small encoder-decoder Transformer model using PyTorch Lightning. The model must support Dutch language for evaluation on the provided test dataset.

Task 2: Decoder-Only Fine-Tuning

Perform fine-tuning with a decoder-only model using techniques such as LoRA-based fine-tuning, instruction tuning, etc.

Task 3: Evaluation

Evaluate models using: - **FLORES-devtest** for general domain quality assessment - **Provided test set** (Dataset_Challenge_1.xlsx) for software domain quality

Provide all relevant metrics to ensure quality aspects are thoroughly addressed.

2. My Approach

2.1 Model Selection

Task	Model	Rationale
Encoder-Decoder	Helsinki-NLP/opus-mt-en-nl (MarianMT)	Pre-trained on EN→NL, 148M parameters, strong baseline
Decoder-Only	bigscience/bloom-560m	Multilingual support including Dutch, LoRA-compatible

2.2 Fine-Tuning Strategy

Encoder-Decoder (MarianMT): - Full parameter fine-tuning - PyTorch Lightning for training orchestration
- AdamW optimizer with linear warmup scheduler - Early stopping based on validation loss

Decoder-Only (BLOOM with LoRA): - Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning
- Only 0.2% of parameters trained (~1.2M out of 560M) - Instruction-tuning format: "Translate English to Dutch:\nEnglish: {src}\nDutch: {tgt}" - PEFT library for LoRA implementation

2.3 Evaluation Metrics

Metric	Library	Purpose
BLEU	sacrebleu	N-gram precision (industry standard)
chrF	sacrebleu	Character-level F-score (important for Dutch compounds)
TER	sacrebleu	Translation Edit Rate (practical measure)

Metric	Library	Purpose
COMET	unbabel-comet	Neural-based semantic quality estimation

3. Repository Structure

```

Domain-Specific-Fine-Tuning/
|
+-- config.py                # All hyperparameters and configuration
+-- data_loader.py           # Data loading for WMT16, FLORES, test set
+-- encoder_decoder_trainer.py # Task 1: MarianMT fine-tuning
+-- decoder_only_trainer.py  # Task 2: BLOOM + LoRA fine-tuning
+-- evaluation.py            # Task 3: BLEU, COMET, chrF, TER
+-- main.py                  # Main execution script
+-- run_evaluation.py         # Quick baseline evaluation
+-- requirements.txt          # Python dependencies
|
+-- data/
|   +-- Dataset_Challenge_1.xlsx # Software domain test set (84 samples)
|
+-- notebooks/
|   +-- 01_domain_finetuning_demo.ipynb # Interactive demonstration
|
+-- outputs/
    +-- evaluation/
        +-- baseline_metrics.json      # Evaluation results
        +-- baseline_translations.xlsx # All translations with references

```

4. Results

4.1 Baseline Evaluation (Software Domain Test Set)

Model: Helsinki-NLP/opus-mt-en-nl

Test Samples: 84 parallel EN→NL sentences

Metric	Score
BLEU	24.45
BLEU-1	42.86
BLEU-2	33.33
BLEU-3	20.00
BLEU-4	12.50
chrF	76.35
TER	34.32

4.2 Sample Translations

Source (English)	Model Output (Dutch)	Reference
Disconnected {1}	Verbindig verbroken {1}	Verbinding verbroken {1} [EXACT MATCH]
Increased contrast	Verhoogd contrast	Verhoogd contrast [EXACT MATCH]
Update window	Venster bijwerken	Updateperiode

4.3 Analysis

- **chrF score of 76.35** indicates strong character-level similarity, important for Dutch compound words
- **TER of 34.32** shows reasonable edit distance between outputs and references
- **BLEU of 24.45** is typical for domain-specific translation without fine-tuning
- Fine-tuning on software domain data would improve domain-specific terminology handling

5. Instructions to Run

5.1 Environment Setup

```
git clone https://github.com/git-bonda108/Domain-Specific-Fine-Tuning.git
cd Domain-Specific-Fine-Tuning
pip install -r requirements.txt
```

5.2 Quick Baseline Evaluation

```
python run_evaluation.py
```

This will: - Load the software domain test set (84 samples) - Run baseline MarianMT model - Compute BLEU, chrF, TER metrics - Save results to `outputs/evaluation/`

5.3 Full Training Pipeline

```
# Run complete pipeline (both models + evaluation)
python main.py --mode all

# Or run individual components
python main.py --mode encoder      # Encoder-decoder fine-tuning only
python main.py --mode decoder      # Decoder-only with LoRA only
python main.py --mode evaluate     # Evaluation only
```

5.4 Configuration

Edit `config.py` to modify: - Model names and paths - Training hyperparameters (learning rate, batch size, epochs) - LoRA configuration (rank, alpha, target modules) - Evaluation settings

6. Technical Implementation Details

6.1 Encoder-Decoder Training (`encoder_decoder_trainer.py`)

```
# Key configuration
model_name = "Helsinki-NLP/opus-mt-en-nl"
learning_rate = 2e-5
batch_size = 16
```

```
num_epochs = 3
warmup_steps = 500
```

6.2 LoRA Configuration (decoder_only_trainer.py)

```
# LoRA parameters
lora_r = 16          # Low-rank dimension
lora_alpha = 32      # Scaling factor
lora_dropout = 0.1   # Dropout for regularization
target_modules = ["query_key_value", "dense", "dense_h_to_4h", "dense_4h_to_h"]
```

6.3 Evaluation (evaluation.py)

```
# Metrics computed
- BLEU (with n-gram breakdown)
- chrF (character-level)
- TER (edit rate)
- COMET (neural quality)
```

7. Files to Review

Priority	File	Description
1	encoder_decoder_trainer.py	Task 1 implementation
2	decoder_only_trainer.py	Task 2 implementation with LoRA
3	evaluation.py	Task 3 metrics implementation
4	outputs/evaluation/baseline_metrics.json	Evaluation results
5	outputs/evaluation/baseline_translations.xlsx	All 84 translations

8. Dependencies

```
torch>=2.0.0
pytorch-lightning>=2.0.0
transformers>=4.35.0
peft>=0.6.0
sacrebleu>=2.3.0
pandas>=2.0.0
openpyxl>=3.1.0
```

9. Hardware Used

- **Device:** Apple Silicon (MPS)
 - **Python:** 3.13
 - **PyTorch:** 2.10.0
-

10. Repository

GitHub: <https://github.com/git-bonda108/Domain-Specific-Fine-Tuning>